Bank of CLI
Part 3

Dennis Keithly
Patrick Guinn
Yashvi Bhagat
Sohan Shrestha
Gian Obenita
Monzerrat Mejia-Mendoza
Gbolahan Adegbola

Project mission

Bank of CLI Fullstack

Transform Bank of CLI into a fully integrated banking application by connecting our React/Angular frontend to a Spring Boot Rest API and PostgreSQL database.

MVPs

01 Authentication & Security
02 REST API Integration
03 Account Management
04 Transactions
05 Transaction History
06 Database Persistence
07 Error Handling & Validation

Contributions

Patrick : React

Authentication

auth • security • register

Yashvi : React

User Experience

transfer • activity • CICD

Monzerrat: Angular

Withdraw

recipiant lookup • angular endpoint impl

Gian : React

Withdraw

withdraw • testing • exception handling • postman

Sohan : React

Deposit

deposit • schema setup • AWS deployment

Gbola: Angular

Withdraw

login • angular impl

Dennis : React

Data Display

transaction history • account overview • postman

Tech Stack

Framework & Language


Java


Spring Boot


React


Angular

DataBase


PostGres SQL

CI/CD


Github Actions

Version Control & Collaboration


Git


GitHub

Security


JWT


Spring Security



Bcrypt

Hosting



AWS


Cloudfront


ECS

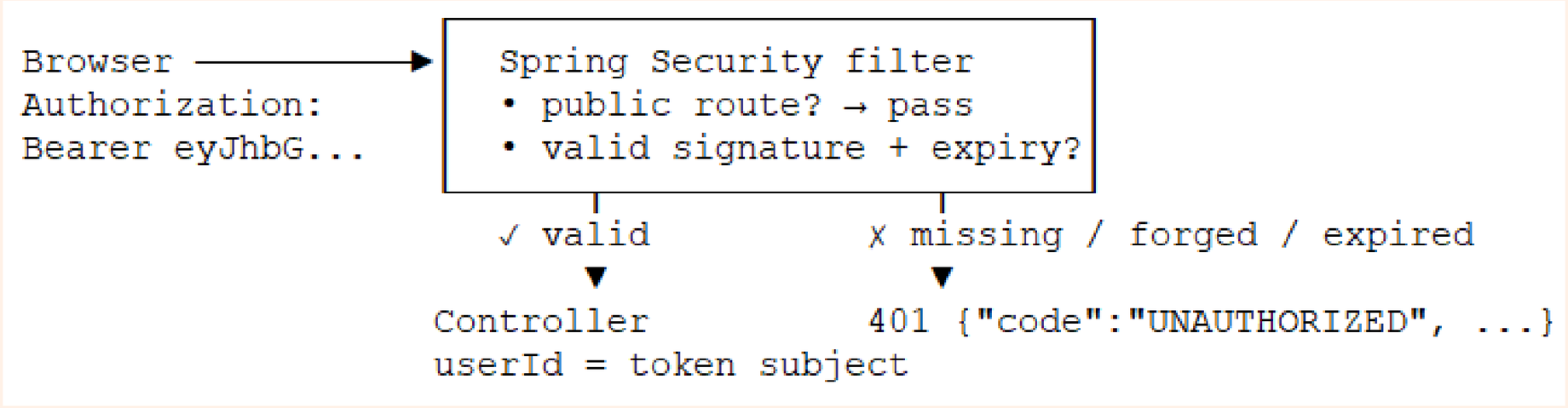
Spring Data



Hibernate

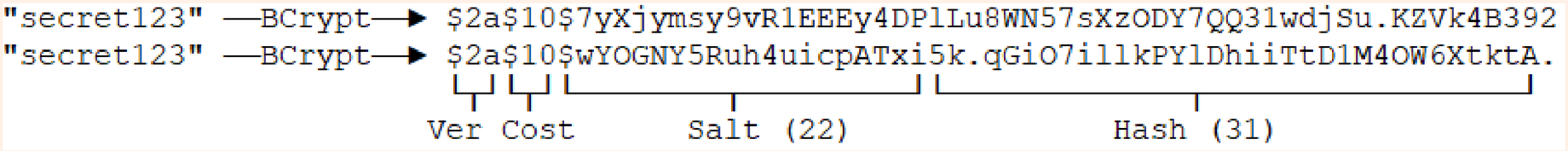
Spring Security

Every request is checked (JWT)



- One gate for every request
- Tokens are signed and expire after 60 minutes
- Identity comes from the token, never the request body

Every password is hashed (BCrypt)



- Same password, different hash
- One-way: a hash can’t be reversed
- Slow by design: cost 10 means 1,024 rounds, which makes guessing expensive

Hibernate/JPA

Flyway

While not a part of the Hibernate cycle, Flyway defined out schema inside of our database to enforce synchronization of data.

JPA

Responsible for transforming our models into entities that could be read by Hibernate. Allows for our repositories to return the data from our modfels

Spring Data JPA

Powers the repository (DAO) layer of our application. Reduces the amount of boilerplate required when making queries to the database.

Hibernate

Process the JPA entities and generates SQL statements from the Spring Data JPA repositories. Returns the JPA annotated entities from what is defined by Flyway in the databasee

Interceptors

Centralized API authentication and session handling

JWT Authentication

Automatically attaches the stored Bearer token to protected API requests.

Session Handling

Clears the token and redirects to login when a protected request returns 401.

Angular Service

Auth Interceptor - Attach JWT

Spring Boot API

401? Clear session → Login

Postman

01

Set up requests

- Login & account requests
- Deposit & withdraw requests
- Transfer API calls

REQUEST

02

Authenticate

- JSON request bodies
- Bearer JWT tokens
- Protected endpoints

AUTHORIZE

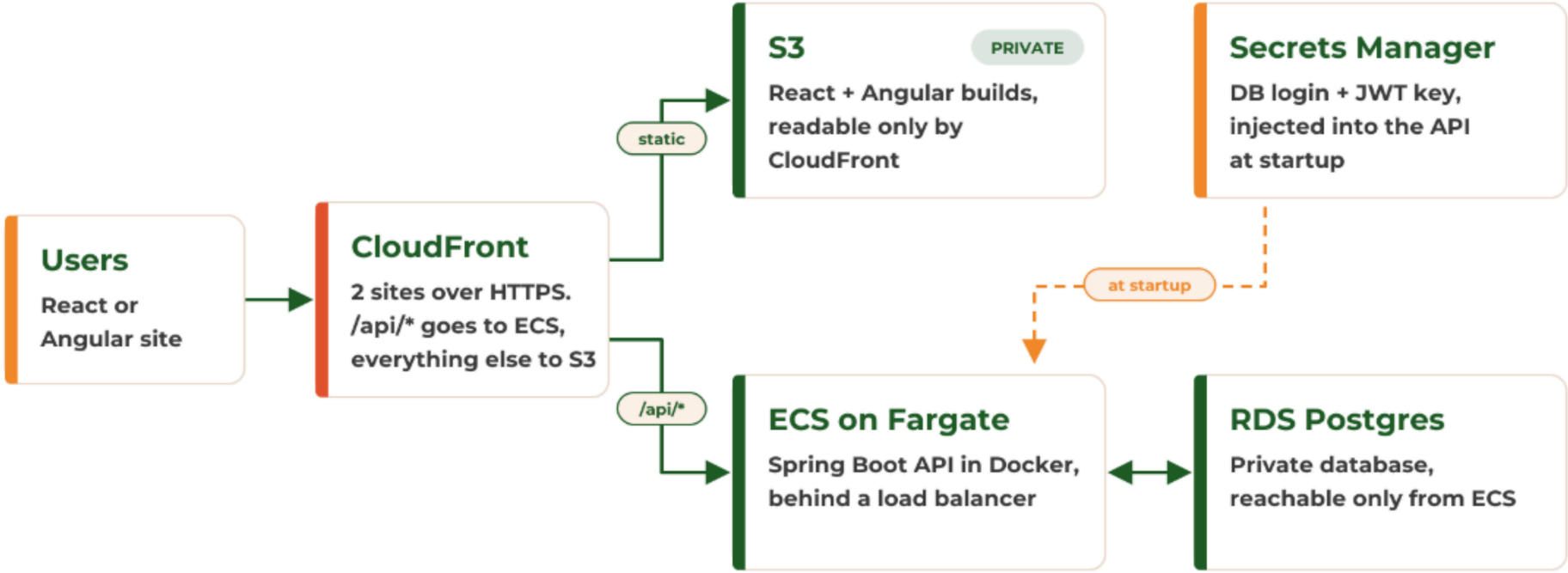
03

Verify results

- HTTP status codes
- Updated balances
- Validation errors

VALIDATE

AWS


Users
React or Angular site
CloudFront
2 sites over HTTPS. /api/* goes to ECS, everything else to S3
static
/api/*
S3
PRIVATE
React + Angular builds, readable only by CloudFront
Secrets Manager
DB login + JWT key, injected into the API at startup
at startup
ECS on Fargate
Spring Boot API in Docker, behind a load balancer
RDS Postgres
Private database, reachable only from ECS

CI/CD: from code to AWS, automatically



CI · Continuous Integration

Every pull request and push to develop / main

- **Backend:** build + tests on real PostgreSQL
- **React:** lint + production build
- **Angular:** unit tests + build
- Parallel jobs; a red check blocks merge

CD · Continuous Deployment

Every push to main

- **Backend:** Docker → ECR → ECS rollout
- **Frontends:** build → S3 → CloudFront
- **OIDC login:** no AWS keys in GitHub
- Old version serves until new is healthy

Demo Flow

1. Register
2. Sign in
3. Review dashboard
4. Deposit
5. Withdraw
6. Transfer
7. Sign out

THANK YOU!